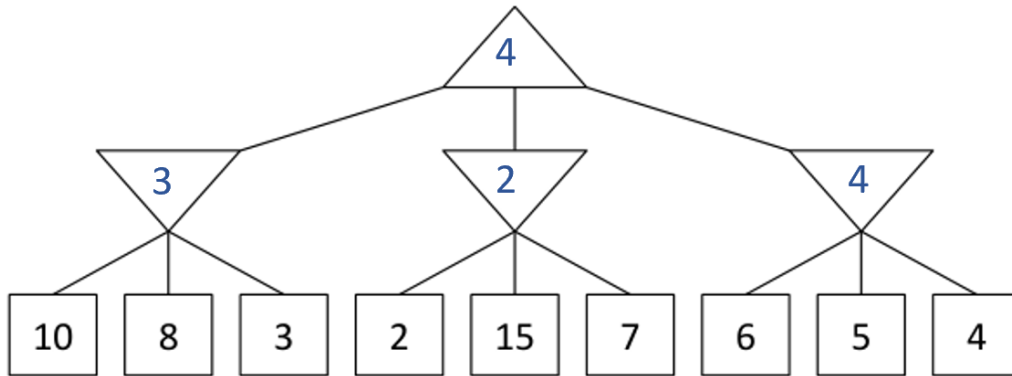


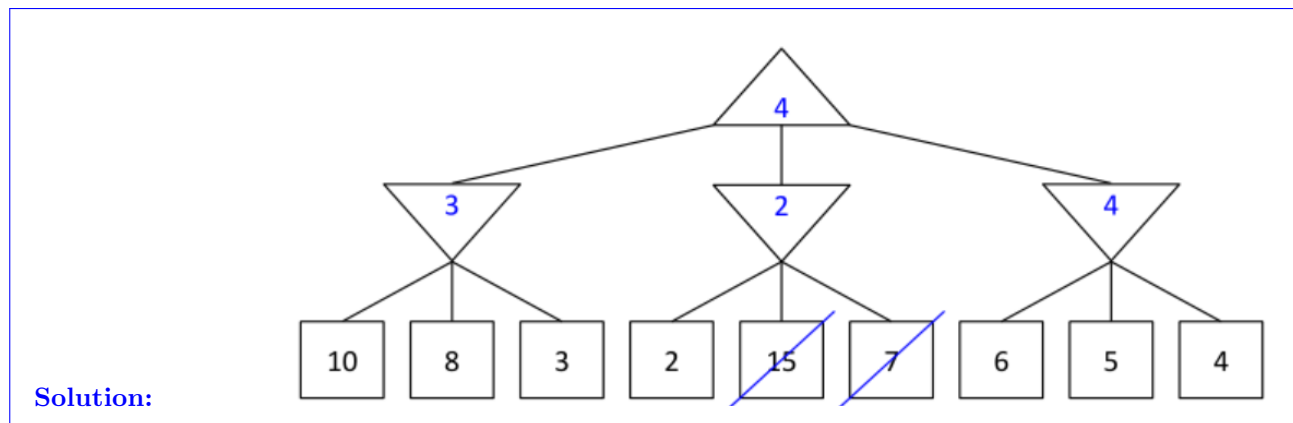
1 A simple Minimax search problem

Consider the zero-sum game tree shown below. Triangles that point up, such as at the top node (root), represent choices for the maximizing player; triangles that point down represent choices for the minimizing player. Assuming both players act optimally, fill in the minimax value of each node.



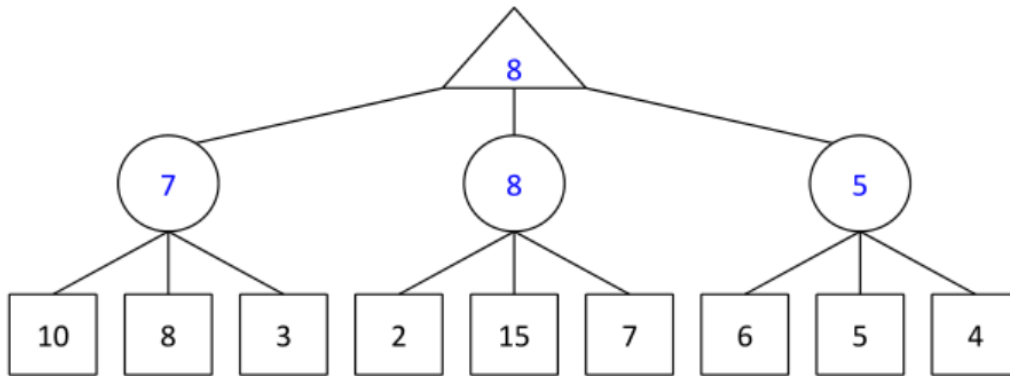
2 α - β pruning

For the same problem of the previous question, which nodes can be pruned from the game tree through alpha-beta pruning? If no nodes can be pruned, explain why not. Assume the search goes from left to right; when choosing which child to visit first, choose the left-most unvisited child.



3 Expectimax

Again, consider the same zero-sum game tree, except that now, instead of a minimizing player, we have a chance node that will select one of the three values uniformly at random. Fill in the expectimax value of each node. The game tree is redrawn below for your convenience.



4 Minimax vs. Expectimax

Consider a simple two-player game with three possible actions from the root node: *Move 1*, *Move 2*, and *Move 3*. Each action can lead to either a deterministic outcome (controlled by the opponent using a minimax strategy) or a probabilistic outcome (controlled by chance nodes). Assume that one of the branches involves a dice roll, which determines the outcome with equal probabilities. How would you modify the minimax algorithm to handle this chance node, and what is the key difference in decision-making between minimax and expectimax at that point?

Solution: To modify the minimax algorithm to handle a chance node (such as a dice roll), you would replace the min and max operators with a weighted sum at the chance nodes. Specifically:

- For each deterministic node (where the opponent moves), the standard minimax logic applies: the agent selects the move that minimizes the worst-case outcome.
- For chance nodes, the agent computes the expected value of the outcomes, considering the probabilities of each result. This is where expectimax differs from minimax: expectimax handles uncertainty by considering the average result, rather than the worst result.

For the dice roll in the example, expectimax would calculate the expected utility as the weighted average of the possible outcomes based on the probabilities of rolling each value. The dice outcomes would all have equal probabilities (since a fair die roll has a $1/6$ chance for each face), and expectimax would sum the utilities of the possible outcomes, each weighted by $1/6$.

5 Again, Minimax vs. Expectimax

Can expectimax ever return a worse decision for the agent compared to minimax? Explain why or why not.

Solution: Yes, expectimax can sometimes return a worse decision for the agent compared to minimax, especially in scenarios where the worst-case outcomes are highly impactful. Minimax is designed to be conservative, preparing for the worst possible outcome, while expectimax chooses actions based on average outcomes. If the worst-case scenario happens to materialize (even if it is unlikely), minimax would have made a safer decision, whereas expectimax might have been overly optimistic. Therefore, expectimax might result in a worse decision in situations where low-probability bad outcomes carry significant risks.

6 Expectimax and α - β

Which nodes can be pruned from the game tree above through alpha-beta pruning? If no nodes can be pruned, explain why not.

Solution: No nodes can be pruned. There will always be the possibility that some leaf further down the branch will have a very high value, which increases the overall average value.

7 α - β pruning step-by-step

Consider now the new zero-sum game tree shown below. Assuming both players act optimally, show step by step how α - β pruning proceed. You need to report the values of α and β , as well as the values (or the bounds)

of the nodes as the search progress.

The search tree has been replicated a few times precisely to let you add all the information stepwise.

